

ARTIFICIAL INTELLIGENCE

35 Important Exam Questions with Easy English Answers + Hinglish Explanation

For beginners

This handbook assumes you are starting AI from zero. Each answer is written so that you can understand it first and then reproduce it in an exam. Diagrams, examples, and memory tricks are included where useful.

How to use this PDF

- Read the Hinglish explanation first if the concept is new.
- For exams, learn the Easy English answer and remember the bold key terms.
- Practice the diagrams for agent architecture and search methods.
- For comparison questions, memorize the final comparison line.

Prepared as beginner-friendly study notes.

Quick Contents

PART 1 - AI BASICS

1. Define AI. What are different task domains or application areas of AI.
2. Explain any five real-life applications of Artificial Intelligence.
3. What are the major challenges involved in developing an AI system? Explain.
4. Differentiate between Artificial Intelligence and conventional programming.

PART 2 - INTELLIGENT AGENTS

5. What is an intelligent agent? Explain the basic structure of an intelligent agent with a diagram.
6. What is a rational agent? Explain with a suitable example.
7. Differentiate between an agent function and an agent program.
8. Explain the role of sensors and actuators in an intelligent agent.
9. Explain the PEAS framework with a suitable example.
10. Construct the PEAS description for a self-driving car.
11. Construct the PEAS description for a chess-playing agent.
12. Describe various types of Agent environment with suitable examples.

PART 3 - TYPES OF AGENTS

13. Explain the Simple Reflex Agent with a suitable example and diagram.
14. Explain the Model-Based Reflex Agent with a suitable example.
15. Explain the Goal-Based Agent with a suitable example and diagram.
16. Explain the Utility-Based Agent with a suitable example.
17. Explain the Learning Agent and its major components.

PART 4 - PROBLEM SOLVING AND STATE SPACE

18. What is a problem-solving agent? Explain its working with a suitable diagram.
19. Explain the concept of problem solving as a search process with suitable example using state space graph.
20. What is problem formulation? Explain its major components.
21. What is a state space? Explain with a suitable example.
22. Differentiate between state space and state-space search.
23. Explain how the 8-puzzle problem can be represented using state-space representation.
24. Formulate the Water Jug problem in terms of initial state, goal state and operators.
25. Formulate the Missionaries and Cannibals problem as a state-space problem.
26. Explain the concept of a goal state and operators using the 8-puzzle problem.
27. Formulate the farmer, wolf, goat, cabbage problem as a state space graph problem.

PART 5 - SEARCH METHODS

28. Explain four metrics to judge a search method's performance.
29. Define Breadth-First Search. Explain its working with a suitable example.
30. Define Depth-First Search. Explain its working with a suitable example.
31. Define Depth-Limited Search. Explain its working with an example.
32. Define Bidirectional Search. Explain its working with a suitable diagram.
33. Write Iterative Deepening algorithm with example. For solving solution in larger state space, which is preferred between DFS with ID and BFS? Explain.
34. Explain Bidirectional Search. Describe how searching simultaneously from the initial state and the goal state can reduce the effective search effort. State the important conditions required for bidirectional search to be applicable.
35. Compare BFS and DFS based on completeness, optimality, time complexity and space complexity.

PART 1 - AI BASICS

1. Define AI. What are different task domains or application areas of AI.

Answer (Easy English)

Artificial Intelligence (AI) is the branch of computer science that builds machines or software able to perform tasks that normally require human intelligence.

AI can learn from data, reason about situations, understand language, recognize patterns, make decisions, and act toward a goal.

- Major task domains / application areas: Natural Language Processing (NLP), Computer Vision, Speech Processing, Robotics, Expert Systems, Game Playing, Planning and Scheduling, Recommendation Systems, Autonomous Vehicles, Medical Diagnosis, and Fraud Detection.

A useful one-line definition for exams is: "AI is the science and engineering of making intelligent machines, especially computer programs, that can perform tasks requiring human-like intelligence."

Diagram

Start with the definition, then write 6-8 application areas.

Hinglish samjho: AI ka simple matlab hai computer ko sirf fixed instructions follow karne ke bajay situation samajh kar intelligent decision lene ke layak banana. Jaise ek normal calculator bas 2+2 karega, but an AI system can understand a sentence, identify a face, or suggest a movie.

2. Explain any five real-life applications of Artificial Intelligence.

Answer (Easy English)

1. Voice Assistants: Siri, Google Assistant, and similar systems use speech recognition and NLP to understand spoken commands and give responses.
2. Recommendation Systems: Shopping and video platforms analyze user behavior and suggest products or content that may be relevant.
3. Medical Diagnosis Support: AI can analyze medical images and patient data to help doctors detect diseases or abnormalities.
4. Self-Driving Vehicles: AI combines cameras, sensors, maps, and decision-making algorithms to detect roads, objects, lanes, and traffic conditions.
5. Fraud Detection: Banks use AI to find unusual transaction patterns and flag potentially fraudulent activity.

Other examples include spam filtering, face recognition, chatbots, smart cameras, and predictive maintenance.

Hinglish samjho: AI humare around already use ho raha hai. Phone me voice assistant, YouTube ka recommendation, bank ka fraud alert, aur medical image analysis - sab AI ke common examples hain. Exam me application + 1 line of working likhna enough hota hai.

3. What are the major challenges involved in developing an AI system? Explain.

Answer (Easy English)

- **Data problem:** AI usually needs enough good-quality data. Poor, biased, or incomplete data can produce poor results.
- **Knowledge representation:** Real-world knowledge is large, complex, and often difficult to represent in a machine-readable form.
- **Reasoning and uncertainty:** Real situations are not always certain. AI must often make decisions with incomplete or noisy information.
- **Learning difficulty:** The system must learn useful patterns without simply memorizing the training data.
- **Generalization:** An AI trained on one situation may fail in a different situation if it cannot generalize.
- **Computational cost:** Large AI systems can require high processing power, memory, and energy.
- **Explainability:** In many applications, users need to know why an AI system made a decision.
- **Ethics and safety:** AI must avoid harmful bias, privacy violations, unsafe behavior, and misuse.

The main problem is that the real world is messy. Human beings can handle incomplete information, exceptions, common sense, and changing situations naturally, but an AI system must be designed and trained to handle them.

Hinglish samjho: Real life clean textbook jaisa nahi hota. Data missing ho sakta hai, situation unexpected ho sakti hai, aur AI ko wrong decision lene se rokna hota hai. Isliye AI banana sirf code likhne ka kaam nahi hai.

4. Differentiate between Artificial Intelligence and conventional programming.

Answer (Easy English)

The main difference is how the system produces behavior.

- **Conventional programming:** Rules are explicitly written by the programmer. Usually, known input + known rules -> output.
- **AI:** The system may learn patterns from data and use them to predict, classify, reason, or decide.
- Conventional systems are often deterministic for the same input and rules, while AI systems may work probabilistically.
- Conventional programming is ideal for well-defined problems such as payroll calculations. AI is useful for problems such as image recognition, speech understanding, and recommendation.

Example: A normal program can calculate salary using fixed formulas. An AI system can learn to predict whether an email is spam from many examples.

Hinglish samjho: Normal programming me programmer pehle hi rules likh deta hai: "agar ye ho to ye karo". AI me rules ko manually likhna har baar possible nahi hota, isliye AI data se pattern seekh sakta hai. Example: spam filter khud learn karta hai kaunse emails suspicious hain.

PART 2 - INTELLIGENT AGENTS

5. What is an intelligent agent? Explain the basic structure of an intelligent agent with a diagram.

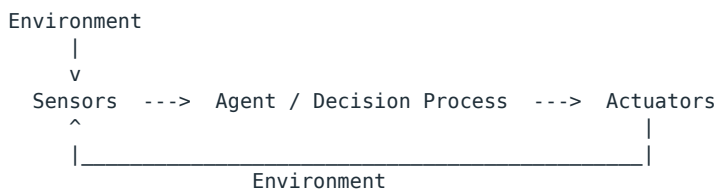
Answer (Easy English)

An intelligent agent is a system that perceives its environment through sensors and acts on the environment through actuators.

The agent receives percepts from the environment, processes them, chooses an action, and then acts.

Examples: a robot, a self-driving car, a game-playing program, or a vacuum-cleaning robot.

Diagram



Hinglish samjho: Agent ko ek smart worker ki tarah samjho. Sensors uski "eyes/ears" hain, agent ka decision-making part "brain" hai, aur actuators uske "hands/legs" hain.

Exam tip: Write: Agent = Sensors + Decision/Processing + Actuators.

6. What is a rational agent? Explain with a suitable example.

Answer (Easy English)

A rational agent is an agent that chooses the action expected to give the best result, based on its percepts, knowledge, and performance measure.

Rational does not mean "always successful". It means the agent chooses the best action it can, using the information available at that time.

Example: A self-driving car sees a red traffic signal. A rational action is to slow down and stop, because stopping improves the safety performance measure.

Hinglish samjho: Rational agent matlab perfect agent nahi. Uske paas jitni information hai, us information ke basis par jo action sabse sensible aur useful hai, wo choose karta hai. Jaise baarish dekhkar umbrella le jaana rational action hai.

7. Differentiate between an agent function and an agent program.

Answer (Easy English)

- Agent function: A mathematical description that maps every possible percept sequence to an action.
- Agent program: The actual software implementation that runs on a machine and computes the action from current percepts or stored information.
- Agent function is an abstract specification; agent program is the implementation.

Example: "For this sequence of road observations, brake" is part of the agent function. The Python/C++ code that checks the sensor data and activates the brake is the agent program.

Hinglish samjho: Function ek theoretical mapping hai - input percept history se output action. Program us mapping ko practically implement karta hai. Matlab function = what the agent should do; program

= how the computer does it.

8. Explain the role of sensors and actuators in an intelligent agent.

Answer (Easy English)

Sensors collect information from the environment. Examples: camera, microphone, temperature sensor, GPS, touch sensor, lidar.

Actuators perform the chosen action in the environment. Examples: wheels, robotic arms, speakers, motors, brakes, and display systems.

The agent depends on sensors for perception and actuators for action. Without sensors it cannot observe; without actuators it cannot affect the environment.

Hinglish samjho: Sensors information andar laate hain; actuators decision ko bahar execute karte hain. Robot ka camera sensor hai, aur uska motor actuator hai.

9. Explain the PEAS framework with a suitable example.

Answer (Easy English)

PEAS is a framework used to describe an intelligent agent task.

- P - Performance Measure: How success is measured.
- E - Environment: Where the agent operates.
- A - Actuators: How the agent can act.
- S - Sensors: How the agent perceives the environment.

Example: Vacuum-cleaning agent: Performance = cleanliness, time, energy use; Environment = rooms and floor; Actuators = move, vacuum, turn; Sensors = dirt sensor, location sensor, bump sensor.

Hinglish samjho: PEAS ko agent ka job description samjho. P batata hai "success kaisa dikhega", E batata hai "kahan kaam hoga", A batata hai "kya actions kar sakta hai", aur S batata hai "kya dekh/sun sakta hai".

10. Construct the PEAS description for a self-driving car.

Answer (Easy English)

- Performance Measure: Safety, avoiding collisions, obeying traffic rules, reaching destination, passenger comfort, travel time, fuel/energy efficiency.
- Environment: Roads, traffic lights, pedestrians, other vehicles, cyclists, weather, road signs, maps, parking areas.
- Actuators: Steering, accelerator, brakes, indicators, horn, lights, gear control.
- Sensors: Cameras, GPS, radar, lidar, ultrasonic sensors, wheel/vehicle sensors.

Hinglish samjho: Self-driving car ko driver ki jagah agent samjho. P = kitna safely aur efficiently destination tak pahucha; E = road world; A = steering/brake etc.; S = camera, GPS, radar etc.

11. Construct the PEAS description for a chess-playing agent.

Answer (Easy English)

- Performance Measure: Win the game, maximize game outcome, avoid losing, obey legal moves, and optionally use time efficiently.
- Environment: Chessboard, opponent, pieces, game rules, clock.

- Actuators: Move a selected chess piece on the board (digital agent: send the move to the game interface).
- Sensors: Board position, opponent move, game state, remaining time.

Hinglish samjho: Chess agent ke paas camera zaroori nahi; digital chess program board state ko directly read karta hai. Uska environment chessboard aur opponent hai, aur actuator selected move hota hai.

12. Describe various types of Agent environment with suitable examples.

Answer (Easy English)

Fully observable vs Partially observable: In fully observable environments, the agent can access all relevant state information. Example: chess. In partially observable environments, some information is hidden or noisy. Example: driving in fog.

Deterministic vs Stochastic: In deterministic environments, the next state is fully determined by the current state and action. Example: 8-puzzle. In stochastic environments, outcomes involve uncertainty. Example: taxi driving in traffic.

Episodic vs Sequential: In episodic environments, each decision is mostly independent. Example: classifying separate images. In sequential environments, current actions affect future states. Example: chess.

Static vs Dynamic: Static environments do not change while the agent is deciding. Example: crossword puzzle. Dynamic environments can change during decision making. Example: road traffic.

Discrete vs Continuous: Discrete environments have countable states/actions. Example: chess. Continuous environments involve values that can vary smoothly. Example: steering a car.

Single-agent vs Multi-agent: Single-agent has one main decision-maker. Example: solving a crossword. Multi-agent includes other agents whose actions matter. Example: football or chess.

Known vs Unknown: In a known environment, the agent knows the rules/outcomes. In an unknown environment, it must learn them.

Hinglish samjho: Environment ke ye properties batati hain problem kitni easy ya hard hai. Chess mostly fully observable, discrete, sequential, multi-agent hai. Driving partially observable, dynamic, continuous, stochastic aur multi-agent ho sakta hai.

PART 3 - TYPES OF AGENTS

13. Explain the Simple Reflex Agent with a suitable example and diagram.

Answer (Easy English)

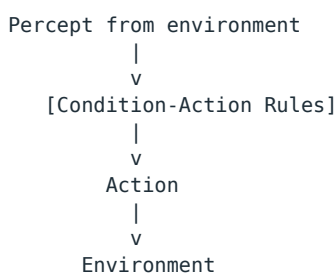
A Simple Reflex Agent selects an action using only the current percept, without considering the past.

It uses condition-action rules such as: IF dirt is detected THEN vacuum.

Example: A simple vacuum agent moves toward dirt or cleans when dirt is present.

It is fast and simple, but it works poorly when the environment is partially observable because it has no memory.

Diagram



Hinglish samjho: Ye agent sirf "abhi kya dikh raha hai?" dekhta hai. Agar abhi dirt mili -> clean. Past me kya hua tha, ye yaad nahi rakhta.

14. Explain the Model-Based Reflex Agent with a suitable example.

Answer (Easy English)

A Model-Based Reflex Agent maintains an internal state that stores useful information about parts of the world that may not be directly visible now.

It uses the current percept, previous internal state, and a model of how the world changes to update its state.

Example: A robot vacuum remembers which rooms it has already cleaned and uses that information when deciding where to go next.

This makes it better than a simple reflex agent in partially observable environments.

Diagram



Hinglish samjho: Simple reflex bhoolakad hai. Model-based agent ke paas internal memory/model hota hai. Agar room abhi visible nahi hai, tab bhi ye pehle ki information ke basis par approximate state maintain kar sakta hai.

15. Explain the Goal-Based Agent with a suitable example and diagram.

Answer (Easy English)

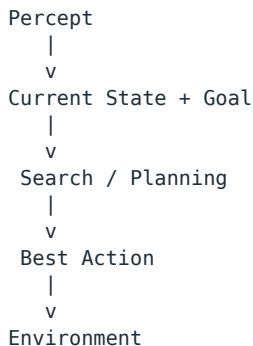
A Goal-Based Agent chooses actions by considering a goal it wants to achieve.

It may search or plan through possible future states before selecting an action.

Example: A navigation system has the goal "reach college" and compares routes before deciding which road to take.

The important idea is that action choice depends on whether it helps the agent reach its goal.

Diagram



Hinglish samjho: Is agent ke paas target hota hai. Sirf present situation nahi dekhta; sochta hai "mujhe kaha pahunchna hai?" aur us goal tak pahunchne ke possible actions evaluate karta hai.

16. Explain the Utility-Based Agent with a suitable example.

Answer (Easy English)

A Utility-Based Agent uses a utility function to compare possible outcomes and choose the one with the highest expected usefulness or preference.

This is useful when several actions can achieve the goal but differ in quality.

Example: A navigation agent may have the goal of reaching a destination, but among several routes it prefers one with less time, less fuel, and less risk.

Goal-based thinking asks "Can I reach the goal?" Utility-based thinking asks "Which reachable outcome is better?"

Hinglish samjho: Goal-based agent bolta hai "destination tak pahunchna hai." Utility-based agent bolta hai "destination tak pahunchna hai, but fastest, safest aur least costly route kaunsa hai?" Utility multiple choices ko rank karne me help karti hai.

17. Explain the Learning Agent and its major components.

Answer (Easy English)

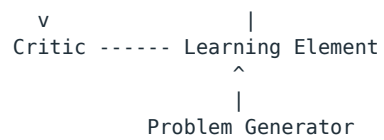
A Learning Agent improves its performance by learning from experience and feedback instead of depending only on fixed rules.

- Four common components are: Performance Element - chooses actions; Learning Element - improves the performance element; Critic - evaluates how well the agent is doing; Problem Generator - suggests exploratory actions to gain useful experience.
- The learning loop is: act -> receive feedback -> learn -> improve future actions.

Example: A game-playing agent plays many games, observes wins/losses, and improves its strategy over time.

Diagram





Hinglish samjho: Learning agent ko student ki tarah samjho. Performance element exam deta hai, critic marks deta hai, learning element mistakes se seekhta hai, aur problem generator naye experiments try karwata hai.

PART 4 - PROBLEM SOLVING AND STATE SPACE

18. What is a problem-solving agent? Explain its working with a suitable diagram.

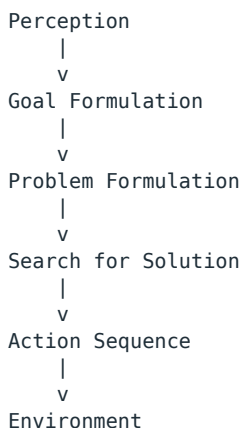
Answer (Easy English)

A problem-solving agent is a goal-based agent that decides what to do by formulating a problem and searching for a sequence of actions that reaches the goal.

Its main steps are: perceive the environment, formulate a goal, formulate the problem, search for a solution, execute the actions, and check the result.

Example: A route-finding agent receives the goal "reach the college", models roads as states and moves as actions, then searches for a route.

Diagram



Hinglish samjho: Problem-solving agent pehle goal decide karta hai, phir problem ko formal form me convert karta hai, phir search karke solution path nikalta hai. Search algorithms isi stage par use hote hain.

19. Explain the concept of problem solving as a search process with suitable example using state space graph.

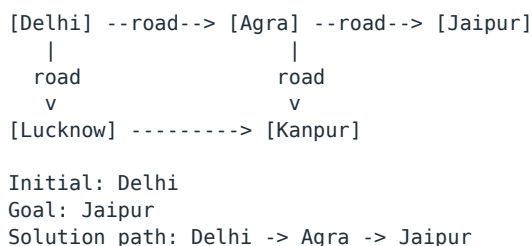
Answer (Easy English)

In AI, many problems can be viewed as a search through a state space. Each node represents a state and each edge represents an action that changes one state into another.

The search starts at the initial state and tries to find a path to a goal state.

Example: Route finding. State = a city, action = travel to a connected city, initial state = Delhi, goal state = Jaipur. A route such as Delhi -> Agra -> Jaipur is a solution path.

Diagram



Hinglish samjho: Isko maze solve karne jaisa socho. Har location ek state hai, road ek edge/action hai, start se goal tak jo valid path milta hai wahi solution hai.

20. What is problem formulation? Explain its major components.

Answer (Easy English)

Problem formulation means converting a real-world problem into a formal form that an AI search algorithm can solve.

- Initial state: Where the agent starts.
- Actions / Operators: What actions are possible.
- Transition model: What state results from each action.
- Goal test: How we know the goal has been reached.
- Path cost: The cost of a sequence of actions, such as distance, time, or money.

Example: For route finding, the initial state is starting city, actions are road choices, transition model gives the next city, goal test checks the destination, and path cost may be total distance.

Hinglish samjho: Real problem ko directly computer ko dena mushkil hota hai. Problem formulation us real problem ko states, actions aur goal ki language me convert karta hai.

21. What is a state space? Explain with a suitable example.

Answer (Easy English)

A state space is the set of all possible states that can be reached from the initial state by applying valid actions.

It can be represented as a graph in which nodes are states and edges are actions.

Example: In the 8-puzzle, every valid arrangement of the 8 tiles and one blank is a state. All reachable arrangements together form the state space.

Hinglish samjho: State space matlab problem ki saari possible situations ka collection. 8-puzzle me har different tile arrangement ek state hai. Un sab states ka complete network state space hai.

22. Differentiate between state space and state-space search.

Answer (Easy English)

- State space: The complete set/graph of possible states and transitions for a problem.
- State-space search: The process/algorithm of exploring that space to find a path from the initial state to a goal state.

Example: Road map = state space. Finding a route on the map using BFS/DFS = state-space search.

Hinglish samjho: Map aur map par route dhundhna alag cheezein hain. Map itself = state space. Map ko explore karke answer find karna = state-space search.

23. Explain how the 8-puzzle problem can be represented using state-space representation.

Answer (Easy English)

The 8-puzzle has 8 numbered tiles and one blank on a 3x3 board.

- State: Any arrangement of the 8 tiles and blank.
- Initial state: The starting arrangement given in the problem.
- Actions: Move the blank up, down, left, or right whenever the move is legal.
- Transition: The selected move swaps the blank with the adjacent tile.

- Goal state: A chosen target arrangement, commonly 1 2 3 / 4 5 6 / 7 8 blank.
- Path cost: Usually 1 for each tile movement, so solution cost equals the number of moves.

Each board arrangement is a node. A legal blank move creates an edge to another arrangement. Search algorithms can then find a path from the initial board to the goal board.

Diagram

Example goal state:

```
1 2 3
4 5 6
7 8 _
```

One legal move:

```
1 2 3      1 2 3
4 _ 6  ->  4 5 6
7 5 8      7 _ 8
           move blank down
```

Hinglish samjho: Har puzzle board ko ek state samjho. Blank ko move karne wala har legal step ek operator hai. Search algorithm different boards ko explore karke goal board tak pahunchta hai.

24. Formulate the Water Jug problem in terms of initial state, goal state and operators.

Answer (Easy English)

Consider the classic 4-liter and 3-liter jugs, with no measuring marks, and unlimited water supply.

- State representation: (x, y), where x is water in the 4L jug and y is water in the 3L jug.
- Initial state: (0, 0).
- Goal state: Any state with exactly 2 liters in the 4L jug, such as (2, 0) or (2, 3).
- Operators: Fill 4L; Fill 3L; Empty 4L; Empty 3L; Pour 4L -> 3L; Pour 3L -> 4L.

Example sequence to obtain 2L in the 4L jug: (0,0) -> (0,3) -> (3,0) -> (3,3) -> (4,2) -> (0,2) -> (2,0). The exact sequence may vary depending on the goal definition.

Hinglish samjho: Do jugs ko ek pair (x,y) ki tarah likhte hain. Har fill/empty/pour ek operator hai. Goal simply exact 2 liters obtain karna hai.

25. Formulate the Missionaries and Cannibals problem as a state-space problem.

Answer (Easy English)

Three missionaries and three cannibals are on the left bank of a river. A boat carries at most two people. The goal is to move everyone safely to the right bank.

- State representation: (M_left, C_left, B), where B indicates the boat side (L or R).
- Initial state: (3,3,L).
- Goal state: (0,0,R).
- Operators: Boat can carry one or two people in allowed combinations: 1M, 2M, 1C, 2C, 1M1C, subject to safety.
- Constraint: On either bank, if missionaries are present, they must not be outnumbered by cannibals.

The search explores only safe states. Unsafe states are rejected, so they do not become valid nodes in the solution search.

Hinglish samjho: State me bas kitne missionaries/cannibals left side par hain aur boat kis side hai, ye track karte hain. Har boat move ek operator hai, but unsafe state allowed nahi hai.

26. Explain the concept of a goal state and operators using the 8-puzzle problem.

Answer (Easy English)

A goal state is a state that satisfies the problem's goal test.

In the 8-puzzle, the goal state can be the arrangement 1 2 3 / 4 5 6 / 7 8 blank.

Operators are legal actions that transform one state into another. The common operators are: move blank up, down, left, or right when possible.

Example: If the blank is in the center, it may be moved in four directions. Each valid move creates a new state.

Hinglish samjho: Goal state = jahan hume pahunchna hai. Operator = ek state se next state me jaane ka valid action. 8-puzzle me blank ki legal movement operators hain.

27. Formulate the farmer, wolf, goat, cabbage problem as a state space graph problem.

Answer (Easy English)

The farmer must carry a wolf, goat, and cabbage across a river. The boat can carry the farmer and at most one item.

- State representation: (F, W, G, C), where each value indicates Left or Right bank.
- Initial state: (L,L,L,L).
- Goal state: (R,R,R,R).
- Operators: Farmer crosses alone, or crosses with wolf, goat, or cabbage, only if the move is safe.
- Safety constraints: Wolf cannot be left alone with goat; goat cannot be left alone with cabbage when farmer is away.

One valid solution path is: take goat across -> return alone -> take wolf across -> bring goat back -> take cabbage across -> return alone -> take goat across.

Diagram

A simple path idea:

```
(L,L,L,L)
|
take Goat
v
(L,L,R,L)
|
return alone
v
...
|
take Goat finally
v
(R,R,R,R)
```

State = (Farmer, Wolf, Goat, Cabbage)

Hinglish samjho: Ye bhi state graph hai. Har node batata hai kaun kis bank par hai. Sirf safe states valid hain. Farmer ko har trip me ensure karna hota hai ki wolf-goat ya goat-cabbage galat combination me unattended na rahe.

PART 5 - SEARCH METHODS

28. Explain four metrics to judge a search method's performance.

Answer (Easy English)

- Completeness: Will the method always find a solution if one exists?
- Optimality: Will it find the best or least-cost solution?
- Time Complexity: How much computation/time is required?
- Space Complexity: How much memory is required?

These four metrics help us compare search strategies such as BFS, DFS, DLS, and Iterative Deepening.

Hinglish samjho: Search algorithm ko sirf "answer mila ya nahi" se judge nahi karte. Ye bhi dekhte hain ki answer best hai ya nahi, kitna time laga, aur kitni memory use hui.

29. Define Breadth-First Search. Explain its working with a suitable example.

Answer (Easy English)

Breadth-First Search (BFS) explores the shallowest nodes first, level by level.

It uses a FIFO queue: the first node inserted is the first node expanded.

Example graph: A → {B,C}; B → {D,E}; C → {F}. BFS order is A, B, C, D, E, F.

For unit step costs, BFS is complete and optimal because the first goal found has the minimum number of steps.

Diagram



BFS order: A → B → C → D → E → F

Hinglish samjho: BFS ko ek floor-by-floor search samjho. Pehle starting room ke direct neighbors, phir unke neighbors. Queue use hoti hai: jo pehle aaya, pehle process hoga.

Exam tip: Remember: BFS = Queue = Level by Level.

30. Define Depth-First Search. Explain its working with a suitable example.

Answer (Easy English)

Depth-First Search (DFS) explores one branch as deeply as possible before backtracking.

It is commonly implemented using a stack or recursion.

Example on the same graph: one possible DFS order is A, B, D, E, C, F, depending on child ordering.

DFS can use much less memory than BFS, but it is not generally optimal and may get stuck exploring a deep or infinite branch without cycle/depth control.

Diagram





One DFS order: A -> B -> D -> E -> C -> F

Hinglish samjho: DFS ko ek maze me ek path ko end tak follow karne jaisa samjho. Dead end mila to backtrack karo aur next path try karo. Stack logic use hota hai.

Exam tip: Remember: DFS = Stack/Recursion = Go Deep First.

31. Define Depth-Limited Search. Explain its working with an example.

Answer (Easy English)

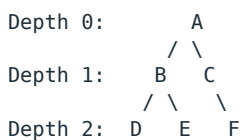
Depth-Limited Search (DLS) is DFS with a fixed depth limit L.

The search does not expand nodes below depth L. This prevents DFS from going infinitely deep.

Example: If the limit is 2, root depth 0, its children depth 1, and grandchildren depth 2 may be explored; nodes at depth 3 are not expanded.

DLS is complete only when a suitable finite limit is at least the depth of a solution. It is not optimal in general.

Diagram



Limit = 2 -> do not expand below depth 2

Hinglish samjho: DFS ki problem kabhi-kabhi bahut neeche chala jaana hai. DLS bolta hai: "maximum 3 levels tak hi jaana hai." Isse uncontrolled deep search se bach sakte hain.

32. Define Bidirectional Search. Explain its working with a suitable diagram.

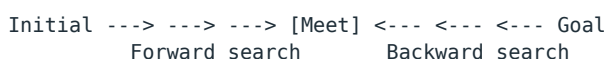
Answer (Easy English)

Bidirectional Search runs two searches simultaneously: one forward from the initial state and one backward from the goal state.

The searches stop when their explored frontiers meet. The final path is built by joining the two partial paths.

It can greatly reduce the effective search effort when the solution depth is d because each side may search roughly d/2 levels.

Diagram



Hinglish samjho: Normal search start se goal ki taraf jaata hai. Bidirectional search dono ends se search karta hai - start se bhi aur goal se bhi. Beech me mil gaye to path combine kar dete hain.

Exam tip: For graphs with reversible actions and a clear goal state, bidirectional search can be very effective.

33. Write Iterative Deepening algorithm with example. For solving solution in larger state space, which is preferred between DFS with ID and BFS? Explain.

Answer (Easy English)

Iterative Deepening Depth-First Search (IDDFS) repeatedly runs Depth-Limited Search with increasing limits: 0, 1, 2, 3, ... until the goal is found.

Pseudo-algorithm: set limit = 0; run DLS(root, limit); if goal not found, increase limit by 1 and repeat.

Example: limit 0 checks only root; limit 1 checks root and children; limit 2 checks grandchildren; continue until the solution is found.

For a large state space with a known finite branching factor and unit step costs, IDDFS is usually preferred over BFS when memory is limited because it uses linear memory like DFS while retaining BFS-like optimality for unit-cost paths. It repeats some work, so time can be higher than BFS.

Diagram

Limit 0: A
Limit 1: A → B, C
Limit 2: A → B, C → D, E, F, ...
Limit 3: expand one more level
...

Hinglish samjho: IDDFS ko "DFS ko baar-baar different depth limit ke saath chalana" samjho. Large state space me BFS ki sabse badi problem memory hai. IDDFS memory ko bahut kam rakhta hai, isliye practical choice ho sakta hai.

Exam tip: Large state space + limited memory: IDDFS is often preferred over BFS for unit-cost problems.

34. Explain Bidirectional Search. Describe how searching simultaneously from the initial state and the goal state can reduce the effective search effort. State the important conditions required for bidirectional search to be applicable.

Answer (Easy English)

Bidirectional search reduces search effort by dividing a depth-d solution into two depth-d/2 searches. Instead of exploring roughly b^d nodes from one side, the idealized work can be closer to $2 \cdot b^{(d/2)}$, where b is the branching factor.

Important conditions: a clearly defined goal state; the ability to generate predecessor states or reverse the actions for the backward search; an efficient test to detect when the two frontiers meet; manageable memory for storing explored/frontier nodes; and a problem where the state representation supports consistent matching from both directions.

For shortest paths with unit step costs, bidirectional BFS is a common choice because it can preserve shortest-path guarantees when implemented correctly.

Hinglish samjho: Do searches dono taraf se chalne par depth half ho jaati hai. Exponential search me depth half karna huge difference create karta hai. But backward search possible hona zaroori hai; har problem me goal se reverse search easy nahi hota.

35. Compare BFS and DFS based on completeness, optimality, time complexity and space complexity.

Answer (Easy English)

Assume branching factor b and shallowest solution depth d . For BFS, time is $O(b^{(d+1)})$ and space is $O(b^{(d+1)})$ in the standard tree-search analysis. For DFS, time is $O(b^m)$, where m is maximum search depth, and space is $O(bm)$ in the standard tree-search analysis.

- BFS: Complete for finite branching factor. Optimal when all step costs are equal. High memory use.
- DFS: Not generally complete in infinite-depth/cyclic spaces unless controlled. Not generally optimal. Much lower memory use than BFS.

Key idea: BFS prefers shallow solutions and uses a queue. DFS prefers deep exploration and uses a stack/recursion.

Diagram

Comparison

BFS: Queue -> Level order -> Complete* -> Optimal* -> High space
DFS: Stack -> Deep first -> Not always -> Not always -> Low space

* under standard conditions: finite branching / equal step costs

Hinglish samjho: BFS ko wide search samjho - poor level store karna padta hai, isliye memory heavy. DFS deep jaata hai, isliye memory kam use karta hai. Exam me complexity ke saath completeness aur optimality zaroor likhna.

Exam tip: A compact memory trick: BFS = complete + optimal (unit cost) but memory heavy; DFS = memory light but not generally complete/optimal.

LAST-MINUTE REVISION SHEET

AI: Machines/software performing tasks that require human-like intelligence.

Agent: Perceives with sensors, acts with actuators.

Rational agent: Chooses the action expected to maximize the performance measure.

PEAS: Performance, Environment, Actuators, Sensors.

Problem formulation: Initial state + actions + transition model + goal test + path cost.

State space: All reachable states and their transitions.

BFS: Queue, level by level, complete, optimal for equal step costs, high memory.

DFS: Stack/recursion, deep first, low memory, not generally complete or optimal.

DLS: DFS with a depth limit.

IDDFS: Repeated DLS with limits 0,1,2,...; low memory and optimal for unit-cost depth solutions.

Bidirectional: Search from both ends and meet in the middle; needs a clear goal and reversible/predecessor generation.

Memory trick: PEAS = Judge the agent. State space = All possibilities. Search = Explore possibilities.